

Projecte d'ESIN

Normativa i Enunciat

Tardor de 2025

Aquest document és llarg però és **imprescindible** que el llegiu íntegrament i amb deteniment, fins i tot per l'alumnat que repeteix l'assignatura. Hi trobareu totes les **instruccions i normes** que heu de seguir per a que el vostre projecte sigui **avaluat positivament**. El professorat de l'assignatura considerarà que **tots els estudiants coneixen i entenen el contingut íntegre** d'aquest document.

Continguts

1	Normativa del projecte	3
2	Enunciat del projecte	5
2.1	L'estratègia FIRST_FIT	6
2.1.1	Inserció de contenidors	7
2.1.2	Àrea d'espera	7
2.1.3	Retirada de contenidors	7
2.1.4	Exemple	7
2.2	L'estratègia LLIURE	9
3	Disseny modular	10

4	La classe ubicacio	12
4.1	Implementació	12
4.2	Especificació de la classe	12
5	La classe contenidor	14
5.1	Decisions sobre les dades	14
5.2	Implementació	14
5.3	Especificació de la classe	14
6	La classe cataleg	17
6.1	Decisions sobre les dades	17
6.2	Implementació	17
6.3	Especificació de la classe	17
7	La classe terminal	20
7.1	Decisions sobre les dades	20
7.2	Implementació	20
7.3	Especificació de la classe	21
8	Errors	25
9	Documentació	26

1 Normativa del projecte

1. **Objectiu i importància del projecte.** Tal com s'indica a la Guia Docent, per a assolir els objectius d'aprenentatge de l'assignatura es considera imprescindible el desenvolupament per part de l'estudiant d'un projecte de programació. Aquest projecte requereix dedicació addicional fora de les hores de laboratori, on es realitzen altres exercicis pràctics orientats a familiaritzar-vos amb l'entorn de treball i el llenguatge de programació C++.
2. **Treball en equip.** El projecte s'ha de realitzar en equips de **dos estudiants**. En cas que un dels integrants abandoni, l'altre ho haurà de comunicar immediatament via correu electrònic a bcasas@cs.upc.edu, i continuarà el projecte en solitari. D'altra banda, només s'autoritzarà la formació d'equips individuals en casos excepcionals on sigui impossible la coordinació o comunicació amb altres estudiants, i s'haurà de justificar mitjançant la corresponent documentació acreditativa.
3. **Entorn i llenguatge de programació.** El projecte s'ha de desenvolupar en el llenguatge C++ (compilador GNU `g++-13.3.0`), dins l'entorn Linux del STIC. Això no impedeix el treball previ en ordinadors personals. De fet, C++ disposa de compiladors per a múltiples plataformes, i la migració a l'entorn del laboratori hauria de ser senzilla, especialment si feu servir GNU/Linux.

Atenció

Poden existir petites incompatibilitats entre compiladors de C++. És obligatori comprovar que el vostre programa funcioni correctament dins l'entorn Linux del STIC, fent, si cal, proves de validació mitjançant connexió remota al servidor `ubiban.epsevg.upc.edu`.

4. **Avaluació.** El projecte serà avaluat d'acord amb:
 - la seva **execució** en l'entorn Linux del STIC, emprant una sèrie de *jocs de prova*;
 - la **qualitat del disseny, implementació i documentació**, incloent-hi les decisions preses, la seva justificació, l'eficiència dels algorismes i estructures de dades, la legibilitat, robustesa i estil de programació.

La documentació s'ha d'entregar **integrada amb el codi font**. No s'admeten entregues en paper.

Hi ha dos tipus de jocs de prova:

- **Públics**, disponibles al Campus Digital (<https://atenea.upc.edu>) amb antelació suficient per provar el projecte.
- **Privats**, utilitzats durant l'avaluació final.

La nota final del projecte es calcula com:

$$P = 0.5E + 0.5D$$

on E és la nota d'execució i D la nota de disseny. Aquesta fórmula només s'aplica si ambdues notes són superiors a 0; en cas contrari, $P = 0$.

Important

Segons el *Manual de laboratori d'ESIN*, capítol G, determinades situacions comporten automàticament una qualificació de 0 en el disseny (i, per tant, en el projecte). La nota d'execució és 3 punts com a mínim si es superen els jocs de prova públics; en cas contrari, és 0. Els jocs de prova privats poden aportar fins a 7 punts addicionals en cas s'hagin superat els jocs de prova públics.

5. **Lliurament.** La data límit d'entrega és el 11 de gener de 2026 a les 23:00. Si no es fa el lliurament dins el termini establert, la nota del projecte serà 0. Els detalls sobre com s'ha de realitzar el lliurament s'indicaran al Campus Digital (<https://atenea.upc.edu>).
6. **Planificació i gestió del temps.** Planifiquen adequadament les diferents fases del projecte: disseny, codificació, depuració, proves i documentació. És freqüent subestimar el temps necessari per a cadascuna d'aquestes tasques.

2 Enunciat del projecte

El propòsit d'aquest projecte és desenvolupar un sistema de gestió automàtica d'una **terminal de contenidors de càrrega**.

Una terminal d'aquest tipus és una gran esplanada organitzada en N fileres de contenidors, cadascuna amb M places. En el problema que ens ocupa, les places són rectangulars –de 10 peus de llarg per 6 d'amplada– i s'alinien una darrera l'altra formant una filera. Entre les fileres hi ha passadissos que permeten la circulació de les grues encarregades de col·locar i retirar els contenidors. Les N fileres constitueixen l'anomenada *àrea d'emmagatzematge*. A més, la terminal disposa d'una *àrea d'espera i maniobres* on es poden emmagatzemar temporalment un petit nombre de contenidors. En aquest projecte, i per simplificar, se suposa que aquesta àrea té capacitat il·limitada.

Tots els contenidors tenen la mateixa amplada i alçada, però poden tenir diferent longitud: n'hi ha de 10, 20 i 30 peus¹. Això implica que un contenidor pot ocupar una, dues o tres places consecutives segons la seva longitud.

Els contenidors es poden apilar uns sobre els altres, sempre i quan la totalitat de la seva base recolzi sobre el terra o sobre altres contenidors del nivell inferior. Ateses les limitacions de pes que poden suportar els contenidors i a les limitacions de les grues de la terminal, només es poden apilar fins un màxim de H pisos. Habitualment, $H = 3$ i mai serà més gran que una certa constant petita H_{MAX} .

Les fileres de la terminal es numeren de 0 a $N - 1$, les places de cada filera de 0 a $M - 1$ i els nivells o pisos es numeren de 0 a $H - 1$. Una *ubicació* és una terna $\langle i, j, k \rangle$ que indica, respectivament, el número de filera, el número de plaça i el pis. La ubicació d'un contenidor és la terna corresponent a la seva plaça més a l'esquerra (és a dir, amb menor j). Cal recordar que els contenidors de 20 i 30 peus ocupen 2 i 3 places, respectivament. El mateix conveni s'aplica per la ubicació d'un forat o grup de places consecutives lliures.

Quan un contenidor nou arriba a la terminal, el sistema ha de trobar automàticament un lloc adequat dins l'àrea d'emmagatzematge. Si hi ha més d'un espai possible on posar el nou contenidor, cal disposar d'una estratègia que decideixi quin triar, intentant complir aquests objectius:

1. que sigui **senzilla** d'implementar i que l'algorisme corresponent sigui eficient en **temps** i ús de **memòria**;
2. que aprofiti l'espai de la terminal de la millor manera possible, minimitzant la **fragmentació**².

¹En realitat, la situació és més complexa i la varietat de mides és més gran, però en aquest projecte s'utilitza un model simplificat.

²S'anomena *fragmentació* al fenomen pel qual no es pot col·locar un contenidor a l'àrea d'emmagatzematge malgrat que, en total, hi hagi espai suficient.

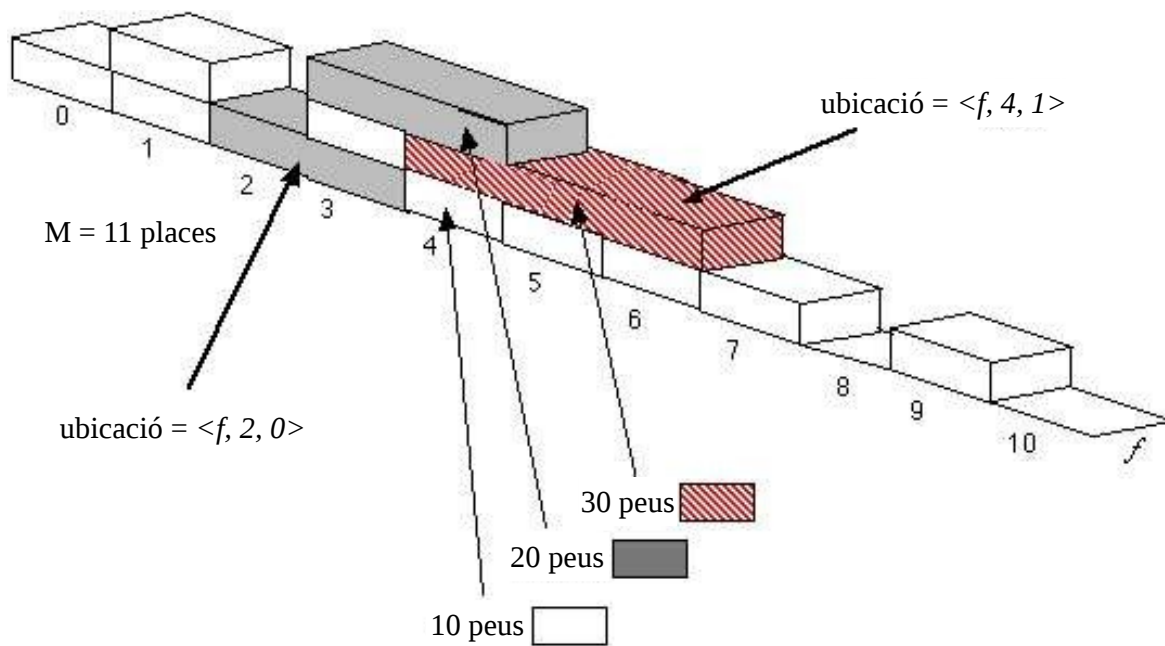


Figura 1: Esquema d'una filera de la terminal de contenidors

A més de decidir on situar cada nou contenidor, una estratègia pot intentar crear espai lliure **reorganitzant** els contenidors ja existents. Si, per qualsevol motiu, no és possible col·locar el nou contenidor dins l'àrea d'emmagatzematge, aquest es deixarà a l'àrea d'espera.

En aquest projecte haureu d'implementar dues estratègies per la inserció i retirada de contenidors de la terminal:

- `FIRST_FIT`, descrita amb detall a la subsecció 2.1.
- `LLIURE`, una estratègia de la vostra invenció, a la subsecció 2.2. .

Per tal de comparar els costos espacials i temporals de diferents estratègies, suposeu que la terminal es troba ocupada entre el 40% i el 60% de la seva capacitat total.

2.1 L'estratègia `FIRST_FIT`

L'estratègia `FIRST_FIT` defineix:

1. quin forat triar per un contenidor quan hi ha diverses opcions disponibles;
2. com gestionar l'àrea d'espera.

► Inserció de contenidors

En inserir un contenidor, si existeixen diversos llocs lliures possibles pel nou contenidor, `FIRST_FIT` tria el “primer” que troba en un hipotètic recorregut per la terminal –filera a filera i plaça a plaça–, és a dir, s’usarà el forat amb el número de filera més baix i en cas d’empat, el forat amb el número de plaça més baix (no pot haver dos forats amb igual nombre de filera i de plaça). Aquesta és la característica que dona nom a l’estratègia.

Si no hi hagués cap forat prou gran per situar el contenidor, `FIRST_FIT` no reorganitza els contenidors de l’àrea d’emmagatzematge: simplement deixa el nou contenidor a l’àrea d’espera.

► Àrea d’espera

L’àrea d’espera es gestiona com una llista de contenidors on els darrers contenidors inserits a la llista són els primers que s’intenten recol·locar. Cada vegada que una inserció té èxit aconseguint col·locar un contenidor en l’àrea d’emmagatzematge, es recorrerà l’àrea d’espera començant pel que es va inserir en últim lloc en l’àrea d’espera i s’intentarà moure’l a l’àrea d’emmagatzematge segons la regla `FIRST_FIT`. Aquest procés es repeteix fins que s’ha examinat completament l’àrea d’espera sense trobar cap contenidor que es pugui moure a l’àrea d’emmagatzematge.

► Retirada de contenidors

Pel que fa a la retirada, l’estratègia `FIRST_FIT` es limita a treure els contenidors que hi hagi per damunt del contenidor a retirar (si n’hi ha), posant-los a l’àrea d’espera. En cas que hi hagi més d’un contenidor a treure, es mourà primer el que tingui la ubicació més petita. Després es recorre la llista intentant reubicar contenidors de l’àrea d’espera a la d’emmagatzematge, seguint un procés idèntic al que s’ha explicat anteriorment per la inserció.

► Exemple

Suposem una terminal (molt petita) amb $N = 1$, $M = 6$ i $H = 3$, inicialment buida. Usarem la notació $i\ m\ n$ per indicar la inserció d’un contenidor amb matrícula m i $10 \cdot n$ peus, i $r\ m$ per indicar la retirada del contenidor de matrícula m . S’han de processar les següents insercions’:

$i\ A\ 3$, $i\ B\ 2$, $i\ C\ 3$, $i\ D\ 1$, $i\ E\ 1$, $i\ F\ 3$, $i\ G\ 1$, $i\ H\ 2$

En aquest punt l'estat de la terminal seria el següent:

```
espera: -    fragmentacio: 1    ops_grua: 8

pis 2  D E G H H
pis 1  B B F F F
pis 0  A A A C C C
```

Ara fem la primera retirada d'un contenidor: `r A`.

Per retirar el contenidor **A 30** cal moure a l'àrea d'espera els contenidors D, E, B, G, H i F, en aquest ordre. Observeu que el contenidor **H 20** no queda damunt d'A, però ha de moure's a l'àrea d'espera per poder moure F. Observeu també que un cop retirats **D 10** i **E 10** es retira B abans que G ja que ambdós estan lliures en aquest moment però B té número de plaça més baix que G. Després de retirar **A 30** es col·loquen tots els contenidors de l'àrea d'espera de nou a l'àrea d'emmagatzematge. Després d'aquesta petició l'estat de la terminal és la següent:

```
espera: -    fragmentacio: 1    ops_grua: 13

pis 2  G E D
pis 1  H H B B
pis 0  F F F C C C
```

A continuació es processen aquestes peticions:

```
r E, r C, i I 1, i J 3, i K 2
```

l'estat final de la terminal és:

```
espera: -    fragmentacio: 1    ops_grua: 30

pis 2  G D J J J
pis 1  H H I K K
pis 0  F F F B B _
```

Cal notar que, en inserir **J 30**, inicialment no hi ha cap espai disponible de manera que queda a l'àrea d'espera; però després de col·locar K, el contenidor **J 30** es pot moure a sobre.

2.2 L'estratègia LLIURE

Els algorismes d'inserció i eliminació de contenidors corresponents a l'estratègia LLIURE són de la vostra elecció. Naturalment, tenen que ser diferents dels definits per FIRST_FIT. La vostra estratègia ha d'especificar:

- quin lloc lliure triar si hi ha més d'una possibilitat;
- en quin ordre es “processa” l'àrea d'espera.

La vostra estratègia pot, si ho considereu oportú, aplicar *reorganitzacions*, és a dir, moure alguns contenidors a l'àrea d'espera per fer lloc a un nou contenidor i després tornar-lo a col·locar a l'àrea d'emmagatzematge, evitant que el contenidor nou hagi de quedar a l'àrea d'espera per problemes de fragmentació. Cal tenir present, però, que els algorismes de reorganització poden arribar a ser molt complicats, el seu cost en temps i espai ser molt elevat i necessitar moltes operacions de grua. A més, encara que el nombre de contenidors a l'àrea d'espera no estigui limitat en aquest projecte, és obvi que moure tots els contenidors de la terminal o tots els contenidors d'una filera a l'àrea d'espera per realitzar una reorganització no és factible en una situació real. Un algorisme de reorganització, si s'usa, mai hauria de moure més que uns pocs contenidors des de l'àrea d'emmagatzematge a l'àrea d'espera.

Observeu que, per exemple, l'estratègia FIRST_FIT **mai** reorganitza contenidors, tot i que procura evitar una fragmentació excessiva. Per exemple, evita col·locar contenidors en el “centre” d'un forat.

Finalment, és important adonar-se que totes les estratègies són *on-line*: les decisions que es prenen, tant en les insercions com en les eliminacions, s'han de basar únicament en l'estat actual de la terminal i no sobre futures (i desconegudes) insercions i eliminacions.

Els tres criteris fonamentals per avaluar la qualitat d'una estratègia són:

- la seva senzillesa conceptual;
- el cost en temps i espai de l'algorisme que la implementa;
- i els valors “mitjans” dels paràmetres de qualitat: fragmentació, longitud de l'àrea d'espera i nombre d'operacions de grua.

Aquests tres criteris no necessàriament es presenten en ordre d'importància.

3 Disseny modular

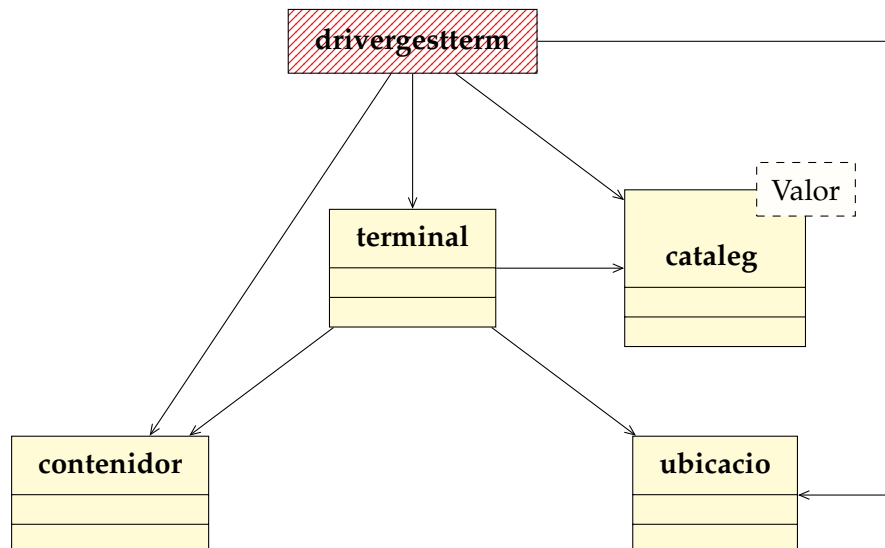


Figura 2: Disseny modular del projecte.

GestTerm consta d'una classe anomenada **terminal** que és l'encarregada de la gestió del terminal. Per tal de realitzar aquesta tasca, aquesta classe usa aquestes altres classes: **ubicacio**, **contenedor** i **cataleg**.

En aquest projecte la vostra tasca és la de dissenyar i implementar les classes **contenedor**, **ubicacio**, **cataleg** i **terminal**.

S'han omès d'aquest diagrama (figura 2) la classe **error** i el mòdul util de la biblioteca **libesin** per claredat, ja que moltes classes i mòduls del diagrama usen aquests mòduls. La classe **error** està documentada en el *Manual de laboratori d'ESIN*, i el mòdul util està documentat on-line en el fitxer `esin/util`. En algunes classes d'aquest projecte s'usa també la classe **string** i la **list** de la biblioteca estàndard de C++. Aquestes relacions d'ús i les classes en qüestió tampoc es mostren a la figura.

També tindreu a la vostra disposició, més endavant, els jocs de proves públics i el mòdul `driver_gestterm`. Aquest mòdul usa a totes les classes que heu d'implementar i permet invocar cada una de les operacions dels diferents mòduls i classes. Amb el `driver` podeu provar, de manera exhaustiva, les diferents classes.

! IMPORTANT: Recordeu que no es pot utilitzar cap classe d'una biblioteca externa en el vostre projecte, exceptuant si en aquesta documentació s'indica el contrari.

En totes les classes s'han d'implementar els mètodes de construcció per còpia, assignació i destrucció davant la possibilitat que useu memòria dinàmica en la classe en qüestió. Si no es fa ús de memòria dinàmica, la implementació d'aquests tres mètodes

és molt senzilla doncs n'hi haurà prou amb imitar el comportament del que serien els corresponents mètodes d'ofici (el destructor no fa "res", i els altres fan còpies atribut per atribut).

Així mateix us proporcionem tots els fitxers capçalera (.hpp) d'aquest disseny modular. No podeu crear els vostres propis fitxers capçalera ni modificar de cap manera els que us donem. Tingueu present que heu de respectar escrupolosament l'especificació de cada classe que apareix en el corresponent .hpp.

! **IMPORTANT:** Un cop implementada cadascuna de les classes és imprescindible que les sotmeteu als vostres jocs de proves. A més, també és fonamental dissenyar amb força detall la representació de cada classe i els seus algorismes sobre paper, i prendre notes de tots els passos seguits abans de començar a codificar. Aquesta informació serà vital de cara a la preparació de la documentació final.

En resum, en aquest projecte les tasques que heu de realitzar i l'ordre que heu de seguir és el següent:

- Implementar la classe ubicacio
- Implementar la classe contenidor
- Implementar la classe cataleg
- Implementar la classe terminal

A l'hora de triar els algorismes i/o les estructures de dades per la representació d'una classe segueix aquestes dues regles d'or:

- 1a. "Davant algorismes de costos temporals asimptòtics idèntics escolliu sempre el més senzill."
- 2a. "No usar exclusivament el criteri del cas pitjor"

4 La classe ubicacio

Una ubicació consisteix en una terna de números $\langle i, j, k \rangle$ corresponents a un número de filera, número de plaça i pis. Hi ha valors del tipus `ubicacio` que tenen significats especials (veure la secció 7): $\langle -1, 0, 0 \rangle$ s'usa per indicar que el contenidor corresponent està a l'àrea d'espera i $\langle -1, -1, -1 \rangle$ per indicar que un contenidor no està a la terminal.

4.1 Implementació

La representació d'aquesta classe es trobarà en el fitxer `ubicacio.rep` i la implementació en el fitxer `ubicacio.cpp`. La documentació haurà de ser mínima, donada la senzillesa de les dades i dels mètodes.

4.2 Especificació de la classe

```
#ifndef _UBICACIO_HPP
#define _UBICACIO_HPP

#include <esin/error>
#include <esin/util>
```

```
class ubicacio {
public:
```

Constructora. Crea la ubicació $\langle i, j, k \rangle$. Genera un error amb codi **UbicacioIncorrecta** si $\langle i, j, k \rangle$ no pertany a $\{ \langle u, v, w \rangle \mid u \geq 0 \vee v \geq 0 \vee w \geq 0 \}$ o a $\{ \langle -1, 0, 0 \rangle, \langle -1, -1, -1 \rangle \}$.

```
ubicacio(int i, int j, int k);
```

Constructora per còpia, assignació i destructora.

```
ubicacio(const ubicacio& u);
ubicacio& operator=(const ubicacio& u);
~ubicacio() noexcept;
```

Consultors. Retornen respectivament el primer, segon i tercer component de la ubicació.

```
int filera() const noexcept;
```

```
int placa() const noexcept;
int pis() const noexcept;
```

Operadors de comparació. L'operador d'igualtat retorna cert si i només si les dues ubicacions tenen la mateixa filera, plaça i pis. L'operador menor retorna cert si i només si la filera del paràmetre implícit és més petit que la d'*u*, o si les dues fileres són iguals i la plaça del paràmetre implícit és més petita que la d'*u*, o si les fileres i les places coincideixen i el pis del paràmetre implícit és més petit que el d'*u*. La resta d'operadors es defineixen consistentment respecte <.

```
bool operator==(const ubicacio &u) const noexcept;
bool operator!=(const ubicacio &u) const noexcept;
bool operator<(const ubicacio &u) const noexcept;
bool operator<=(const ubicacio &u) const noexcept;
bool operator>(const ubicacio &u) const noexcept;
bool operator>=(const ubicacio &u) const noexcept;
```

Gestió d'errors

```
static constexpr int UbicacioIncorrecta = 10;

private:
    #include "ubicacio.rep"
};
#endif
```

5 La classe contenidor

Cada objecte de la classe contenidor representa un contenidor físic de la terminal i es caracteritza per dos atributs principals: una *matrícula o identificador* i una *longitud*. La matrícula està formada per una seqüència de lletres majúscula i dígit (p.e. AX02B7Z8) i la longitud pot ser 10, 20 o 30 peus.

En una aplicació real, cada contenidor disposaria de molta més informació –com ara la data d’entrada a la terminal, la data prevista de sortida, la descripció de la càrrega, el pes, les dades d’identificació del propietari, les dades de procedència, el model del contenidor, etc.– però en el marc d’aquest projecte simplifiquem la representació i ens limitarem als atributs essencials.

5.1 Decisions sobre les dades

- Les matrícules es representaran mitjançant cadenes de caràcters (`string`). Tot i que la longitud de la cadena no està limitada, una matrícula només serà vàlida si compleix els següents requisits:
 - conté exclusivament lletres majúscules (de la A a la Z) o dígit;
 - no és buida

Queden exclosos caràcters especials com la Ñ, la Ç o les vocals accentuades.³

- La longitud del contenidor es representarà mitjançant un enter positiu (`unsigned int`), el qual indicarà la mida del contenidor en peus.

5.2 Implementació

La representació d’aquesta classe es trobarà en el fitxer `FILE contenidor.rep` i la implementació en el fitxer `FILE contenidor.cpp`. La documentació haurà de ser mínima, donada la senzillesa de les dades i dels mètodes.

5.3 Especificació de la classe

³Les matrícules només poden contenir caràcters amb codi ASCII entre 65 ('A') i 90 ('Z') o dígit entre 48 ('0') i 57 ('9').

```
#ifndef _CONTENIDOR_HPP
#define _CONTENIDOR_HPP

#include <string>
#include <esin/error>
#include <esin/util>
```

```
using std::string;
using util::nat;
```

```
class contenidor {
public:
```

Constructora. Crea un contenidor amb matrícula m i longitud l . Es produeix un error amb codi **MatriculaIncorrecta** si m no és una seqüència de un o més caràcters, formada exclusivament per lletres majúscules i dígit. Es produeix un error amb codi **LongitudIncorrecta** si l no pertany a 10, 20, 30.

```
    contenidor(const string &m, nat l);
```

Constructora per còpia, assignació i destructora.

```
    contenidor(const contenidor &u);
    contenidor& operator=(const contenidor &u);
    ~contenidor() noexcept;
```

Consultors. Retornen respectivament la longitud i la matrícula del contenidor.

```
    nat longitud() const noexcept;
    string matricula() const noexcept;
```

Operadors de comparació. L'operador d'igualtat retorna cert si i només si els dos contenidors contenen la mateixa matrícula i longitud. L'operador menor retorna cert si i només si la matrícula del paràmetre implícit és més petit en ordre alfabètic que la de c o si les dues matrícules són iguals i la longitud del paràmetre implícit és més petita que la de c . La resta d'operadors es defineixen consistentment respecte a $<$.

```
    bool operator==(const contenidor &c) const noexcept;
    bool operator!=(const contenidor &c) const noexcept;
    bool operator<(const contenidor &c) const noexcept;
    bool operator<=(const contenidor &c) const noexcept;
    bool operator>(const contenidor &c) const noexcept;
    bool operator>=(const contenidor &c) const noexcept;
```

Gestió d'errors

```
static constexpr int MatriculaIncorrecta = 20;
static constexpr int LongitudIncorrecta = 21;

private:
    #include "contenedor.rep"
};
#endif
```


6 La classe cataleg

Un objecte de la classe `cataleg` representa una col·lecció de parells $\langle clau, valor \rangle$ en la que no hi ha dos parells amb igual clau.

En el mètode constructor d'aquesta classe s'indica el nombre d'elements que com a màxim s'inseriran en el catàleg. En qualsevol cas el catàleg ha de permetre emmagatzemar elements més enllà del màxim indicat.

6.1 Decisions sobre les dades

- Les claus són *strings* no buits.
- El tipus `Valor` ha de disposar del constructor per còpia i de l'operador assignació.

6.2 Implementació

La representació d'aquesta classe es trobarà en el fitxer `FILE cataleg.rep` i la implementació en el fitxer `FILE cataleg.t`.

Observacions: No es pot emprar cap classe de la biblioteca estàndard de C++, exceptuant la classe `string` per representar les claus.

6.3 Especificació de la classe

```
#ifndef _CATALEG_HPP
#define _CATALEG_HPP
#include <string>
#include <esin/error>
#include <esin/util>

using std::string;
using util::nat;

template <typename Valor>
class cataleg {
public:
```

Constructora. Crea un catàleg buit on *numelems* és el nombre aproximat d'elements que com a màxim s'inseriran al catàleg.

```
explicit cataleg(nat numelems);
```

Constructora per còpia, assignació i destructora.

```
cataleg(const cataleg& c);  
cataleg& operator=(const cataleg& c);  
~cataleg() noexcept;
```

Mètode modificador. Insereix el parell $\langle \text{clau}, \text{valor} \rangle$ indicat. En cas que la clau k ja existeixi en el catàleg actualitza el valor associat. Genera un error amb codi **ClauStringBuit** en cas que la clau sigui l'string buit.

```
void assig(const string &k, const Valor &v);
```

Elimina del catàleg el parell que té com a clau k . En cas que la clau k no existeixi en el catàleg genera un error amb codi **ClauInexistent**.

```
void elimina(const string &k);
```

Retorna cert si i només si la clau k existeix dins del catàleg; fals en cas contrari.

```
bool existeix(const string &k) const noexcept;
```

Retorna el valor associat a la clau k ; si no existeix cap parell amb clau k llavors genera un error amb codi **ClauInexistent**. Exemple:

```
cataleg<int> ct(91);  
...  
int n = ct["dia"];
```

```
Valor operator[](const string &k) const;
```

Retorna el nombre d'elements que s'han inserit en el catàleg fins aquest moment.

```
nat quants() const noexcept;
```

Gestió d'errors

```
static constexpr int ClauStringBuit = 30;  
static constexpr int ClauInexistent = 31;
```

```
private:
    #include "cataleg.rep"
};
#include "cataleg.t"
#endif
```

7 La classe terminal

Aquesta classe ofereix dues operacions “principalment”: `insereix_contenedor` i `retira_contenedor`, a més d’un cert nombre de mètodes auxiliars i el mètode constructor. La primera d’aquestes operacions s’utilitza quan un nou contenidor arriba a la terminal i la segona quan un contenidor abandona la terminal.

Per retirar un contenidor *c* (que està a l’àrea d’emmagatzematge) s’han de moure a l’àrea d’espera tots els contenidors que estiguin sobre el contenidor en qüestió, eliminar el contenidor *c* de la terminal i intentar recol·locar contenidors de l’àrea d’espera a l’àrea d’emmagatzematge. Si el contenidor que es vol retirar està a l’àrea d’espera, s’elimina de manera immediata.

7.1 Decisions sobre les dades

- `HMAX` és una constant.
- Les operacions `on` i `longitud` reben un `string` qualsevol com a paràmetre.

7.2 Implementació

La representació d’aquesta classe es trobarà en el fitxer `terminal.rep` i la implementació en el fitxer `terminal.cpp`.

Observacions: Es pot emprar la classe `list` de la biblioteca estàndard de C++, però únicament per representar i gestionar l’àrea d’espera del terminal i implementar el mètode `area_espera`.

Atès que les operacions `insereix_contenedor` i `retira_contenedor` poden involucrar el moviment de varis contenidors des de l’àrea d’espera a l’àrea d’emmagatzematge, resultaria ineficient o molt complicat garantir que la o les estructures de dades que representen la terminal quedin en el seu estat original si es produís un error de memòria dinàmica durant un pas intermedi de l’execució. Per aquesta raó, i excepcionalment, la vostra implementació d’aquests dos mètodes no tindrà que deixar necessàriament inalterada l’estructura de dades en el cas que es produeixi un error de manca de memòria dinàmica. Si es produeix algun altre tipus d’error, la regla sí que s’aplica, ja que aquests altres errors poden ser fàcilment verificats abans de procedir a la modificació de l’estructura.

7.3 Especificació de la classe

```
#ifndef _TERMINAL_HPP
#define _TERMINAL_HPP
#include <list>
#include <string>
#include <esin/error>
#include <esin/util>

#include "cataleg.hpp"
#include "contenedor.hpp"
#include "ubicacio.hpp"

using std::string;
using std::list;
using util::nat;

class terminal {
public:
    static constexpr nat HMAX = 7;
    enum class estrategia { FIRST_FIT, LLIURE };

```

Constructora. Crea una terminal buida amb n fileres de m places cadascuna, i una alçada màxima d'apilament h ; a més fixa l'estratègia d'inserció i retirada dels contenidors respecte el paràmetre st . Genera un error amb codi **NumFileresIncorr**, **NumPlacesIncorr** o **AlcadaMaxIncorr** si $n = 0, m = 0, h = 0, h > HMAX$ o un error amb codi **EstrategiaIncorr** si st no pertany a {FIRST_FIT, LLIURE}.

```
terminal(nat n, nat m, nat h, estrategia st);
```

Constructora per còpia, assignació i destructora.

```
terminal(const terminal& b);
terminal& operator=(const terminal& b);
~terminal() noexcept;
```

Col·loca el contenidor c en l'àrea d'emmagatzematge de la terminal o en l'àrea d'espera si no troba lloc en l'àrea d'emmagatzematge usant l'estratègia prefixada en el moment de crear la terminal. Si el contenidor c es col·loca en l'àrea d'emmagatzematge pot succeir que un o més contenidors de l'àrea d'espera puguin ser moguts a l'àrea d'emmagatzematge. En aquest cas es mouran els contenidors de l'àrea d'espera a l'àrea d'emmagatzematge seguint l'ordre que indiqui l'estratègia que s'està usant. Finalment, genera un error amb codi **MatriculaDuplicada** si ja existís a la terminal un contenidor amb una matrícula idèntica que la del contenidor c .

```
void insereix_contenedor(const contenidor &c);
```

Retira de la terminal el contenidor c la matrícula del qual és igual a m . Aquest contenidor pot estar a l'àrea d'emmagatzematge o a l'àrea d'espera. Si el contenidor estigués a l'àrea d'emmagatzematge llavors s'hauran de moure a l'àrea d'espera tots els contenidors que siguin necessaris per netejar la part superior de c , s'hauran de retirar possiblement diversos contenidors, començant pel contenidor sense cap altre a sobre amb el número de plaça més baix (més a l'esquerra) i així successivament (veure exemple amb detall a la subsecció 2.1). Un cop s'hagi eliminat el contenidor indicat, s'intenta moure contenidors de l'àrea d'espera a l'àrea d'emmagatzematge, seguint l'ordre que indiqui l'estratègia que s'està usant. Genera un error amb codi **Matriculainexistent** si a la terminal no hi ha cap contenidor la matrícula del qual sigui igual a m .

```
void retira_contenidor(const string &m);
```

Retorna la ubicació $\langle i, j, k \rangle$ del contenidor la matrícula del qual és igual a m si el contenidor està a l'àrea d'emmagatzematge de la terminal. Si el contenidor està a l'àrea d'espera retorna la ubicació $\langle -1, 0, 0 \rangle$. Si no existeix cap contenidor que tingui una matrícula igual a m retorna la ubicació $\langle -1, -1, -1 \rangle$. Cal recordar que si un contenidor té més de 10 peus, la seva ubicació correspon a la plaça que tingui el número de plaça més petit.

```
ubicacio on(const string &m) const noexcept;
```

Retorna la longitud del contenidor la matrícula del qual és igual a m . Genera un error amb codi **Matriculainexistent** si no existeix un contenidor a la terminal la matrícula del qual sigui igual a m .

```
nat longitud(const string &m) const;
```

Retorna la matrícula del contenidor que ocupa la ubicació $u = \langle i, j, k \rangle$ o la cadena buida si la ubicació està buida. Genera un error amb codi **UbicacioNoMagatzem** si $i < 0, i \geq n, j < 0, j \geq m, k < 0$ o $k \geq h$, o sigui si $\langle i, j, k \rangle$ no identifica una ubicació vàlida de l'àrea d'emmagatzematge. Cal observar que si m , obtinguda amb `t.contenidor_ocupa(u, m)`, és una matrícula (no la cadena buida) pot succeir que $u \neq t.on(m)$, ja que un contenidor pot ocupar diverses places i la seva ubicació es correspon amb la de la plaça ocupada amb número de plaça més baix.

```
void contenidor_ocupa(const ubicacio &u, string &m) const;
```

Retorna el nombre de places de la terminal que en aquest instant només hi cabrien un contenidor de 10 peus, però no un de més llarg. Per exemple, la filera de la figura 1 de l'enunciat contribuirà amb 7 unitats a la fragmentació total (corresponen a les ubicacions $\langle f, 0, 1 \rangle, \langle f, 1, 2 \rangle, \langle f, 2, 1 \rangle, \langle f, 7, 1 \rangle, \langle f, 8, 0 \rangle, \langle f, 9, 1 \rangle$ i $\langle f, 10, 0 \rangle$).

```
nat fragmentacio() const noexcept;
```

Retorna el número d'operacions de grua realitzades des del moment de creació de la terminal. Es requereix d'una operació de grua per moure un contenidor des de l'àrea d'espera a l'àrea d'emmagatzematge o viceversa. També es requereix d'una operació de grua per inserir o retirar directament un contenidor de l'àrea d'emmagatzematge. En canvi no requereix cap operació de grua inserir o retirar directament un contenidor de l'àrea d'espera.

```
nat ops_grua() const noexcept;
```

Retorna la llista de les matrícules de tots els contenidors de l'àrea d'espera de la terminal, en ordre alfabètic creixent.

```
void area_espera(list<string> &l) const noexcept;
```

Retorna el número de fileres de la terminal.

```
nat num_fileres() const noexcept;
```

Retorna el número de places per filera de la terminal.

```
nat num_places() const noexcept;
```

Retorna l'alçada màxima d'apilament de la terminal.

```
nat num_pisos() const noexcept;
```

Retorna l'estratègia d'inserció i retirada de contenidors de la terminal.

```
estrategia quina_estrategia() const noexcept;
```

Gestió d'errors

```
static constexpr int NumFileresIncorr = 40;  
static constexpr int NumPlacesIncorr  = 41;  
static constexpr int AlcadaMaxIncorr  = 42;  
static constexpr int EstrategiaIncorr = 43;  
static constexpr int MatriculaDuplicada = 44;  
static constexpr int MatriculaInexistent = 45;  
static constexpr int UbicacioNoMagatzem = 46;
```

```
private:
```

```
#include "terminal.rep"  
};  
#endif
```


8 Errors

Aquest fitxer conté els missatges d'error usats en la gestió d'errors.

```
10 ubicacio Ubicacio incorrecta.  
  
20 contenidor Matricula incorrecta.  
21 contenidor Longitud incorrecta.  
  
30 cataleg Clau es l'string buit.  
31 cataleg Clau inexistent.  
  
40 terminal Numero de fileres incorrecte.  
41 terminal Numero de places incorrecte.  
42 terminal Alcada maxima incorrecta.  
43 terminal Estrategia incorrecta.  
44 terminal Matricula duplicada.  
45 terminal Matricula inexistent.  
46 terminal Ubicacio no pertany al magatzem.
```

9 Documentació

Els fitxers lliurats han d'estar degudament documentats. És molt important descriure amb detall i precisió la representació escollida en el fitxer `.rep`, justificant les eleccions fetes, així com les operacions de cada classe. És especialment important explicar amb detall les representacions i els motius de la seva elecció enfront a possibles alternatives, i els algorismes utilitzats.

El cost en temps i en espai és freqüentment el criteri determinant en l'elecció, per la qual cosa s'hauran de detallar aquests costs en la justificació (sempre que això sigui possible) per a cada alternativa considerada i per a l'opció finalment escollida. A més caldrà detallar en el fitxer `.cpp` el cost de cada mètode públic i privat.

En definitiva heu de:

- comentar adequadament el codi, evitant comentaris inútils i superflus
- indicar, en la mesura que sigui possible, el cost dels mètodes de les classes (tant públics com privats)
- descriure amb detall i precisió la representació escollida i justifiqueu l'elecció respecte d'altres.

Un cop enviats els fitxers per via electrònica, aquests seran impresos per a la seva avaluació. No haureu d'imprimir-los vosaltres. No haureu de lliurar cap altra documentació. Per tal d'unificar l'aspecte visual del codi fem servir una eina de *prettyprinting* anomenada `astyle`. Podeu comprovar els resultats que produeix el *prettyprinter* mitjançant la comanda

```
% astyle --style=kr -s2 < fitxer.cpp > fitxer.formatejat
```

i a continuació podeu convertir-lo en un PDF per visualitzar-lo o imprimir-lo

```
% a2ps fitxer.formatejat -o - | ps2pdf - fitxer.pdf
```